Pipelined RISC-V Processor Design Phase-II

Batta Venkata Shashank
2024102057

Ganipisetty Nischal
2024102070

Regalla Kowshikh Reddy
2024102053

March 10, 2026

Team: **NRS I-11 Processor**

1 Introduction

In the First phase of the project, a sequential RISC-V processor was implemented. In a sequential processor, each instruction is executed completely before the next instruction begins. While this approach simplifies the design and ensures correct execution, it results in low throughput because only one instruction is processed at a time. As a consequence, several hardware components remain idle during different stages of instruction execution, leading to inefficient utilization of processor resources.

Pipelining divides the execution of an instruction into multiple stages so that different instructions can be processed simultaneously at different stages. In this project, a five-stage pipelined RISC-V processor is designed consisting of the following stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). By overlapping these stages, the processor can execute multiple instructions concurrently, thereby increasing the overall instruction throughput.

However, pipelining introduces several challenges known as pipeline hazards. These hazards occur when the normal flow of instruction execution is disrupted. The main types of hazards include data hazards, control hazards, and structural hazards. Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed. Control hazards arise due to branch instructions that change the flow of execution. Structural hazards occur when multiple instructions compete for the same hardware resource at the same time.

A forwarding unit is used to pass results directly from later pipeline stages to earlier stages, reducing delays caused by data dependencies. A hazard detection unit is implemented to identify situations where forwarding alone is not sufficient, such as load-use dependencies, and temporarily stall the pipeline. Control hazards caused by branch instructions are handled using a static branch prediction strategy that assumes branches are not taken and flushes incorrect instructions when required.

Through the use of pipeline registers, forwarding logic, and hazard detection mechanisms, the pipelined processor improves execution efficiency while ensuring correct instruction execution.

2 Five-Stage Pipeline Architecture

2.1 Instruction Fetch (IF)

The Instruction Fetch stage is responsible for fetching the instruction from the instruction memory. The Program Counter (PC) holds the address of the current instruction. During this stage, the instruction located at the address specified by the PC is read from memory. After fetching the instruction, the PC is updated to point to the next instruction, typically by incrementing it by 4 since each RISC-V instruc-

tion is 32 bits long.

2.2 Instruction Decode (ID)

In the Instruction Decode stage, the fetched instruction is interpreted to determine the type of operation that needs to be performed. The instruction fields such as opcode, source registers, and destination register are extracted. The required register values are read from the register file. Additionally, immediate values are generated if the instruction requires them. Control signals needed for the later stages are also generated during this stage.

2.3 Execute (EX)

The Execute stage performs the main computation required by the instruction. The Arithmetic Logic Unit (ALU) carries out operations such as addition, subtraction, logical operations, or address calculations. For branch instructions, the branch condition is evaluated and the branch target address is computed in this stage.

2.4 Memory Access (MEM)

In the Memory Access stage, the processor interacts with the data memory. If the instruction is a load instruction, data is read from memory. If it is a store instruction, data is written to memory. For other instructions that do not require memory access, this stage simply passes the ALU result to the next stage.

2.5 Write Back (WB)

The Write Back stage is the final stage of the pipeline. In this stage, the result produced either by the ALU or by the data memory is written back to the register file. This updates the destination register so that the result can be used by subsequent instructions.

3 Pipeline Hazards and Forwarding

In a pipelined processor, a new instruction begins execution before the previous instruction has completed. This overlapping of instructions allows multiple instructions to be processed simultaneously at different stages of the pipeline, thereby improving the overall throughput of the processor.

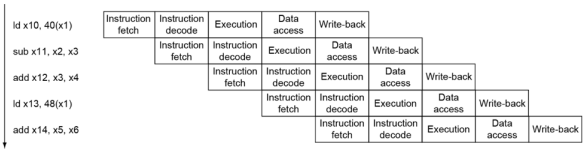

Figure 1: All the instructions are independent

If all instructions are independent, pipelining works efficiently without any problems. Each instruction progresses through the pipeline stages without interfering with the execution of other instructions. However, problems arise when instructions depend on the results of previous instructions. In such cases, an instruction may require a value that has not yet been produced by an earlier instruction, leading to incorrect execution if the pipeline proceeds normally.

Consider the following example:

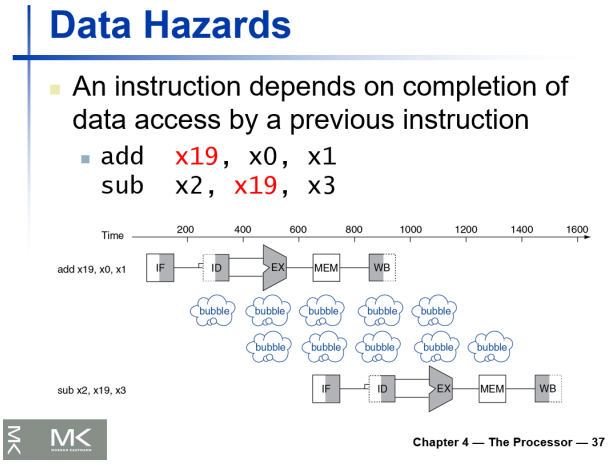

Figure 2: Data Hazard

The first instruction computes the value of register `x19`. The second instruction requires this value as one of its operands. In a pipelined processor, the second instruction may reach the execute stage before the first instruction has written the result back to the register file. As a result, the second instruction may read an incorrect value of `x19`. This dependency causes a problem in the pipeline execution. This type of hazard is called a **Data Hazard**.

Another situation occurs when the processor encounters a branch instruction.

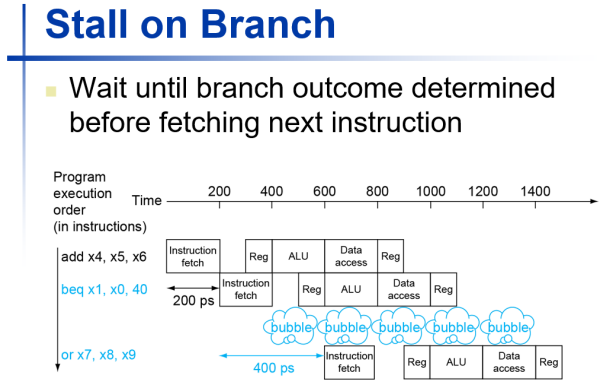

Figure 3: Control Hazard

```
add x4, x5, x6
beq x1, x0, 40
or  x7, x8, x9
```

When the branch instruction is fetched, the processor does not immediately know whether the branch condition is true or false. The branch decision is determined only after the comparison is performed in the execute stage. Meanwhile, the processor may continue fetching the next instruction. If the branch is taken, the fetched instruction may not belong to the correct execution path and must be discarded, which introduces bubbles in the pipeline. This situation is known as a **Control Hazard**.

One technique used to reduce delays caused by data dependencies is **data forwarding**. Instead of waiting for the result to be written back to the register file, the result can be directly forwarded from a later stage of the pipeline to an earlier stage where it is required.

Consider the following example:

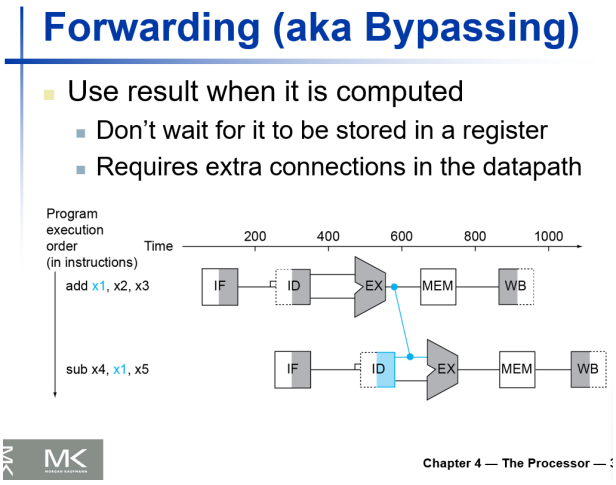

Figure 4: Forwarding

```
add x1, x2, x3
sub x4, x1, x5
```

The first instruction generates the value of `x1`. The second instruction requires this value to perform its computation. In forwarding, the result produced by

the ALU in the first instruction is directly sent to the ALU input of the second instruction without waiting for it to be written back to the register file. This allows the dependent instruction to continue execution without introducing additional pipeline stalls.

4 Pipeline Registers and Control Signal Propagation

In a pipelined processor, pipeline registers are placed between every two consecutive stages to store intermediate data and control signals. These registers ensure that each instruction carries its required information as it moves through the pipeline stages.

In the five-stage pipeline, four pipeline registers are used:

- IF/ID
- ID/EX
- EX/MEM
- MEM/WB

Each pipeline register stores only the information required for the remaining stages of the pipeline. As the instruction progresses through the pipeline, some control signals are no longer needed and therefore are not passed to the next pipeline register.

4.1 IF/ID Pipeline Register

The IF/ID pipeline register is placed between the Instruction Fetch (IF) stage and the Instruction Decode (ID) stage. It stores the fetched instruction (`instr_in`) and the corresponding program counter value (`pc_in`) so that they can be used in the decode stage during the next clock cycle.

The control signal `IF_ID_Write` is used to stall the pipeline by preventing the register from updating its contents. The signal `IF_ID_Flush` is used to handle control hazards by inserting a bubble instruction when a branch is taken. The outputs `instr_out` and `pc_out` provide the stored instruction and PC value to the decode stage.

Only these signals are stored because they are the only outputs generated in the fetch stage that are required by the decode stage. Control signals and register data are generated later in the pipeline and therefore are not stored in this register.

4.2 ID/EX Pipeline Register

The ID/EX pipeline register is placed between the Instruction Decode (ID) stage and the Execute (EX) stage. It stores the outputs generated in the decode stage so that they can be used during the execute stage in the next clock cycle.

The inputs `pc_in`, `reg1_in`, and `reg2_in` store the program counter value and the register operands read from the register file. The signal `imm_in` stores the immediate value generated from the instruction. The register fields `rs1_in`, `rs2_in`, and `rd_in` are also stored so that they can be used later for operations such as forwarding and hazard detection.

In addition to the data signals, the ID/EX register stores the control signals generated by the control unit. These include `RegWrite`, `MemRead`, `MemWrite`, `MemtoReg`, `ALUSrc`, `Branch`, and `ALUOp`. These control signals determine how the instruction will be executed in the subsequent stages of the pipeline.

The signal `ID_EX_Flush` is used to handle hazards by clearing the control signals and effectively inserting a bubble into the pipeline. The outputs of this register provide the stored data and control signals to the execute stage for further processing.

4.3 EX/MEM Pipeline Register

The EX/MEM pipeline register is placed between the Execute (EX) stage and the Memory (MEM) stage. It stores the outputs generated in the execute stage so that they can be used in the memory stage during the next clock cycle.

The input `alu_result_in` stores the result produced by the ALU during the execute stage. The signal `write_data_in` contains the value that may be written to data memory in the case of store instructions. The destination register `rd_in` is also stored so that the correct register can be updated in the write-back stage.

In addition to the data signals, the EX/MEM register stores the control signals required for the later stages of the pipeline. These include `RegWrite`, `MemRead`, `MemWrite`, and `MemtoReg`. These signals determine whether the instruction will read from memory, write to memory, or write the result back to the register file.

The outputs of this register pass the ALU result, memory write data, destination register number, and the required control signals to the memory stage so that the appropriate memory operation can be performed.

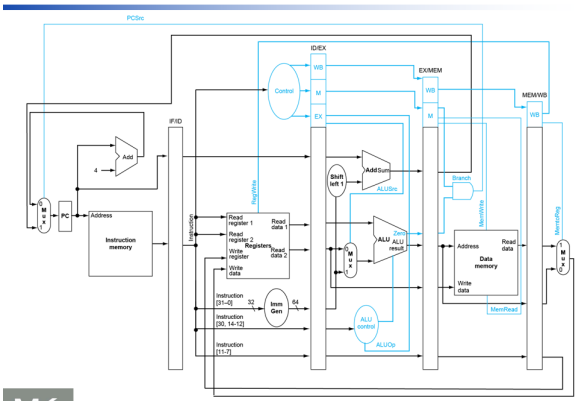

Figure 5: Pipeline Registers and controls

4.4 MEM/WB Pipeline Register

The MEM/WB pipeline register is placed between the Memory (MEM) stage and the Write Back (WB) stage. Its purpose is to store the outputs generated during the memory stage so that they can be used during the write-back stage in the next clock cycle.

The signal `mem_data_in` stores the data read from the data memory for load instructions. The signal `alu_result_in` stores the result produced by the ALU, which may be written back to the register file for arithmetic or logical instructions. The destination register `rd_in` is also stored so that the correct register can be updated during the write-back stage.

In addition to the data signals, the MEM/WB register stores the control signals `RegWrite` and `MemtoReg`. The signal `RegWrite` determines whether the result should be written to the register file, while `MemtoReg` selects whether the value written back comes from the memory output or the ALU result.

The outputs of this register provide the required data and control signals to the write-back stage so that the correct value can be written into the destination register.

4.5 Destination Register Propagation

The destination register is originally obtained from the instruction during the decode stage. However, since multiple instructions are present in the pipeline simultaneously, the correct destination register must travel along with the instruction through the pipeline registers. Therefore, the register number is passed from ID/EX to EX/MEM and finally to MEM/WB. The register file uses the value stored in the MEM/WB register during the Write Back stage. If the register number were taken directly from the current instruction, the result might be written to the wrong register because that instruction would belong to a different pipeline stage.

5 Data Hazards

A data hazard occurs when an instruction depends on the result produced by a previous instruction that has not yet completed its execution. In a pipelined processor, multiple instructions execute simultaneously in different stages of the pipeline. Because of this overlap, the required data may not yet be available when the dependent instruction reaches the execute stage.

5.1 Dependency and Forwarding

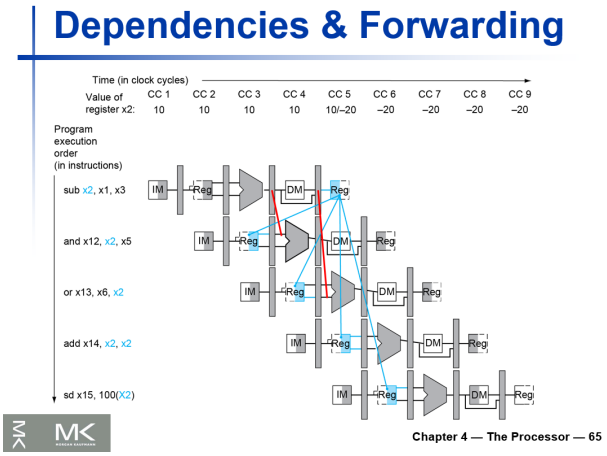

Figure 6: dependency

Consider the instruction sequence shown in figure

```
sub x2, x1, x3
and x12, x2, x5
or  x13, x6, x2
add x14, x2, x2
sd  x15, 100(x2)
```

In this sequence, the first instruction produces the value of register `x2`. The following instructions depend on this value. Since the result is generated in the Execute stage but written back only in the Write Back stage, the subsequent instructions may require the value earlier.

When the instruction `sub x2, x1, x3` finishes the Execute stage, the ALU result is stored in the EX/MEM pipeline register. When the next instruction `and x12, x2, x5` reaches the Execute stage, it requires the value of `x2`. Instead of waiting for the value to be written back to the register file, the value is forwarded directly from the **EX/MEM pipeline register to the Execute stage**.

When the instruction `sub x2, x1, x3` progresses further in the pipeline and reaches the Write Back stage, its ALU result is stored in the MEM/WB pipeline register. When the instruction `or x13, x6, x2` reaches the Execute stage, it requires the value of `x2`. Since the result of the first instruction is now available in the MEM/WB pipeline register, the value is forwarded directly from the **MEM/WB pipeline register to the Execute stage**.

5.2 Double Data Hazard

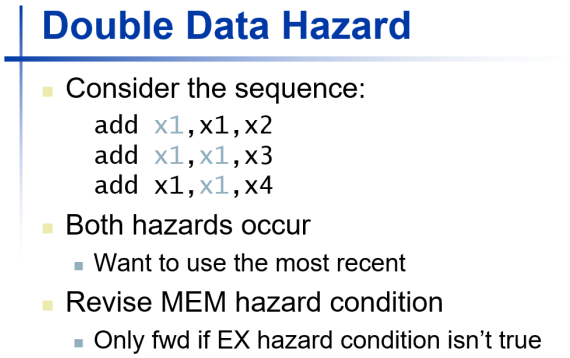

Figure 7: doublehazard

Another situation occurs when multiple consecutive instructions depend on the same register value. Consider the sequence shown in Figure

```
add x1, x1, x2
add x1, x1, x3
add x1, x1, x4
```

Here, every instruction writes to register `x1` and the next instruction immediately depends on that updated value. This creates multiple overlapping dependencies in the pipeline.

When the third instruction executes, two possible forwarded values may exist: one from the EX/MEM pipeline register and another from the MEM/WB pipeline register. The processor must select the most recent value of the register.

Therefore, the forwarding unit first checks the EX/MEM pipeline register. If the destination register in EX/MEM matches the source register of the current instruction and the control signal `RegWrite = 1`, the value is forwarded from the EX/MEM stage.

If this condition is not satisfied, the forwarding unit checks the MEM/WB pipeline register. If the destination register matches and `RegWrite = 1`, the value is forwarded from MEM/WB. This ensures that the most recently computed value is used and prevents the processor from using an outdated result.

5.3 Load-Use Data Hazard

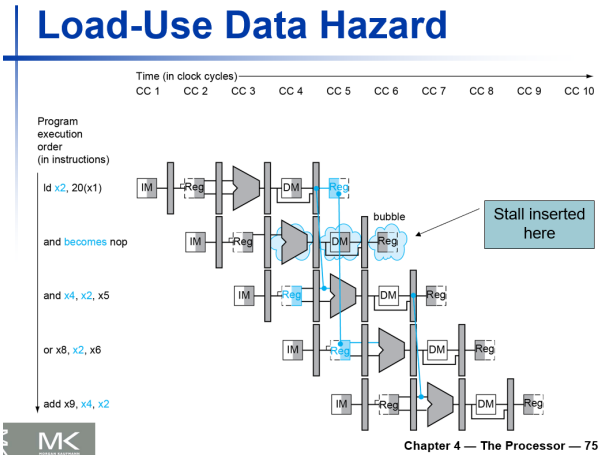

Figure 8: Load-use data Hazard

A special type of data hazard occurs when a load instruction is immediately followed by an instruction that uses the loaded value. This situation is shown in Figure

```
ld  x2, 20(x1)
and x4, x2, x5
or  x8, x2, x6
add x9, x4, x2
```

In this case, the value loaded from memory becomes available only after the Memory stage. However, the next instruction requires the value during the Execute stage. Since the data is not yet available, forwarding alone cannot resolve the dependency.

To handle this situation, the processor inserts a **pipeline stall** (or bubble). This temporarily delays the dependent instruction until the correct data becomes available. After the stall cycle, the pipeline continues execution normally.

5.4 Store Data Forwarding

Store data forwarding occurs when a store instruction requires a value that is produced by a previous instruction that has not yet completed the write-back stage. Without forwarding, the store instruction may read an incorrect value from the register file.

Consider the following example:

```
add x5, x1, x2
sd  x5, 0(x3)
```

The first instruction generates the value of register `x5`. The store instruction must write this value to memory. However, during pipelined execution the value may not yet be written back to the register file when the store instruction reaches the memory stage.

To resolve this dependency, the processor forwards the result directly from the EX/MEM or MEM/WB pipeline register to the data memory input. This technique ensures that the correct value is written to memory without introducing pipeline stalls.

5.5 Forwarding Conditions

The forwarding unit detects hazards by comparing the destination register of instructions in later pipeline stages with the source registers of the instruction currently in the Execute stage.

5.5.1 EX Hazard (EX/MEM → EX)

An EX hazard occurs when the instruction in the EX/MEM pipeline register produces a result that is required by the instruction currently in the Execute stage.

The forwarding condition is given by:

ForwardA:
if $(EX/MEM.RegWrite = 1)$
$\&(EX/MEM.RegisterRd \neq 0)$
$\&(EX/MEM.RegisterRd = ID/EX.RegisterRs)$
then $ForwardA = 10$

ForwardB:
if $(EX/MEM.RegWrite = 1)$
$\&(EX/MEM.RegisterRd \neq 0)$

$\&(EX/MEM.RegisterRd = ID/EX.RegisterRt)$ then $ForwardB = 10$

5.5.2 MEM Hazard (MEM/WB → EX)

The MEM hazard occurs when the required value is available in the MEM/WB pipeline register and must be forwarded to the Execute stage.

ForwardA:
if $(MEM/WB.RegWrite = 1)$
$\&(MEM/WB.RegisterRd \neq 0)$
$\&\neg(EX/MEM.RegWrite\&(EX/MEM.RegisterRd \neq 0)$
$\&(EX/MEM.RegisterRd = ID/EX.RegisterRs))$
$\&(MEM/WB.RegisterRd = ID/EX.RegisterRs)$
then $ForwardA = 01$

ForwardB:
if $(MEM/WB.RegWrite = 1)$
$\&(MEM/WB.RegisterRd \neq 0)$
$\&\neg(EX/MEM.RegWrite\&(EX/MEM.RegisterRd \neq 0)$
$\&(EX/MEM.RegisterRd = ID/EX.RegisterRt))$
$\&(MEM/WB.RegisterRd = ID/EX.RegisterRt)$
then $ForwardB = 01$

Forwarding Conditions

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Figure 9: Forwarding Conditions

5.6 TestCases:

5.6.1 Testcase 1:

```
addi x1, x0, 5
add  x3, x1, x0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

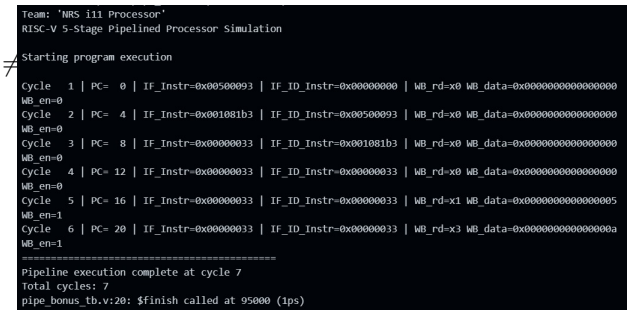

Figure 10: Output

The second instruction requires `x1` as `rs1`. Since the result of the first instruction is available in the EX/MEM pipeline register, the value is forwarded from EX/MEM to the Execute stage.

Forwarding condition:
if $(EX/MEM.RegWrite = 1)$
$\&(EX/MEM.RegisterRd \neq 0)$
$\&(EX/MEM.RegisterRd = ID/EX.RegisterRs)$
then $ForwardA = 10$

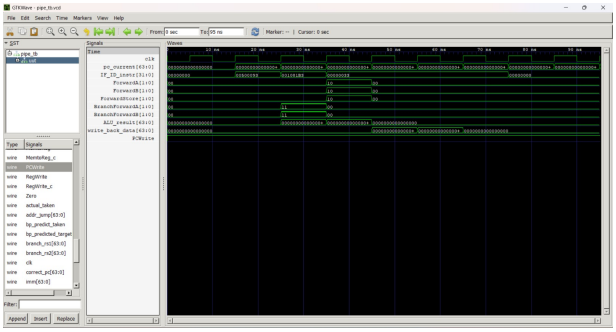

Figure 11: GTK Wave

5.6.2 TestCase 2:

```
addi x1, x0, 5
add  x3, x0, x1
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation

Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00500093 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x001001b3 | IF_ID_Instr=0x00500093 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x00000033 | IF_ID_Instr=0x001001b3 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x1 WB_data=0x0000000000000005
WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x3 WB_data=0x0000000000000005
WB_en=1
========================================
Pipeline execution complete at cycle 7
Total cycles: 7
pipe_bonus_tb.v:20: $finish called at 95000 (1ps)
```

Figure 12: Output

The second instruction uses the value of `x1` as the second source register (`rs2`). The result of the first instruction is available in the EX/MEM pipeline register when the second instruction reaches the Execute stage. Therefore, the value is forwarded from the EX/MEM pipeline register to the ALU input.

Forwarding condition:

if $(EX/MEM.RegWrite = 1)$
$\&(EX/MEM.RegisterRd \neq 0)$
$\&(EX/MEM.RegisterRd = ID/EX.RegisterRt)$
then $ForwardB = 10$

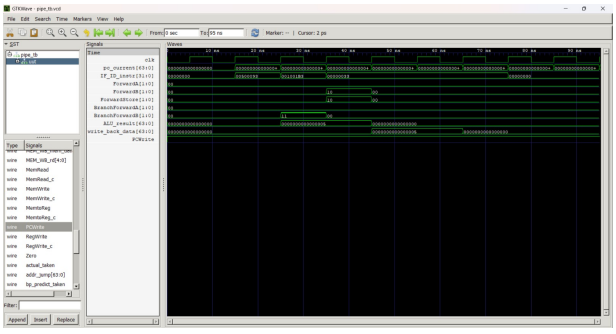

Figure 13: GTK Wave

5.6.3 TestCase 3:

```
addi x1, x0, 5
addi x2, x0, 7
add  x3, x1, x0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation

Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00500093 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x00700113 | IF_ID_Instr=0x00500093 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x001081b3 | IF_ID_Instr=0x00700113 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x00000033 | IF_ID_Instr=0x001081b3 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x1 WB_data=0x0000000000000005
WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x2 WB_data=0x0000000000000007
WB_en=1
Cycle   7 | PC= 24 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x3 WB_data=0x000000000000000a
WB_en=1
========================================
Pipeline execution complete at cycle 8
Total cycles: 8
pipe_bonus_tb.v:20: $finish called at 105000 (1ps)
```

Figure 14: Output

The third instruction requires the value of `x1` as the first source register (`rs1`). At this time the result of the first instruction has already reached the MEM/WB pipeline register. Therefore, the value is forwarded from MEM/WB to the Execute stage.

Forwarding condition:

if $(MEM/WB.RegWrite = 1)$
$\&(MEM/WB.RegisterRd \neq 0)$
$\&(MEM/WB.RegisterRd = ID/EX.RegisterRs)$
then $ForwardA = 01$

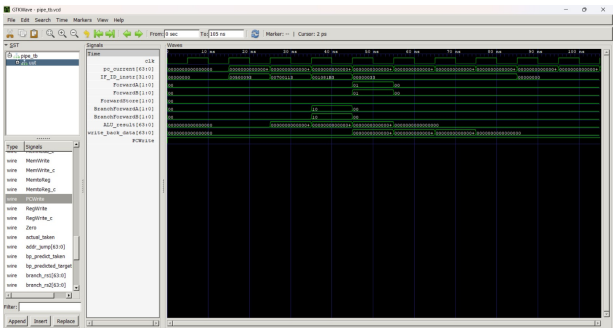

Figure 15: GTK Wave

5.6.4 TestCase 4:

```
addi x1, x0, 5
addi x2, x0, 7
add  x3, x0, x1
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

Figure 16: Output

The third instruction requires the value of `x1` as the second source register (`rs2`). At this time the result of the first instruction is available in the MEM/WB pipeline register. Therefore, the value is forwarded from MEM/WB to the Execute stage.

Forwarding condition:

if $(MEM/WB.RegWrite = 1)$ $\&(MEM/WB.RegisterRd \neq 0)$ $\&(MEM/WB.RegisterRd = ID/EX.RegisterRt)$ then $ForwardB = 01$

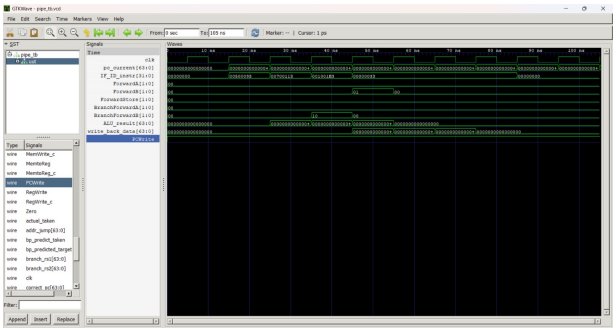

Figure 17: GTK Wave

5.6.5 TestCase 5:

```
  addi x1, x0, 8
addi x2, x0, 42
sd   x2, 0(x1)
ld   x3, 0(x1)
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

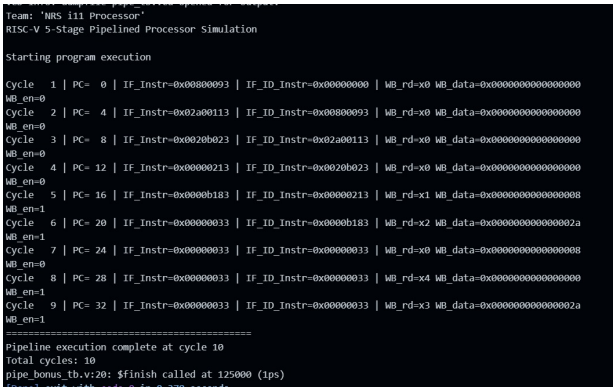

Figure 18: Output

The store instruction requires the value of `x2` as the data to be

written into memory. The value of `x2` is produced by the previous instruction and is available in the EX/MEM pipeline register. Therefore, the store data is forwarded from the EX/MEM register to the memory stage.

Forwarding condition:

if $(EX/MEM.RegWrite = 1)$ $\&(EX/MEM.RegisterRd = ID/EX.RegisterRs2)$ then $ForwardStore = 10$

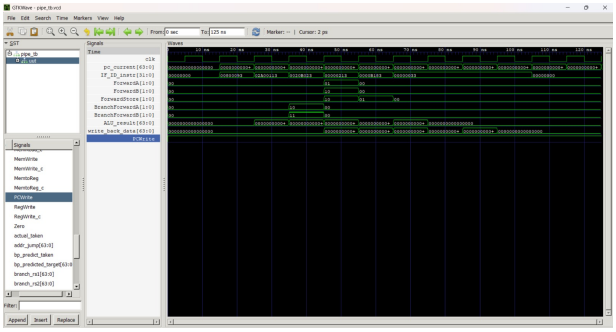

Figure 19: GTK Wave

5.6.6 TestCase 6:

```
addi x1, x0, 8
addi x2, x0, 99
sd   x2, 0(x1)
ld   x3, 0(x1)
sd   x3, 16(x1)
addi x5, x0, 0
ld   x4, 16(x1)
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation
Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00800093 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x06300113 | IF_ID_Instr=0x00800093 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x0020b023 | IF_ID_Instr=0x06300113 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x0000b183 | IF_ID_Instr=0x0020b023 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x0030b823 | IF_ID_Instr=0x0000b183 | WB_rd=x1 WB_data=0x0000000000000008
WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00000293 | IF_ID_Instr=0x0030b823 | WB_rd=x2 WB_data=0x0000000000000063
WB_en=1
Cycle   7 | PC= 24 | IF_Instr=0x0100b203 | IF_ID_Instr=0x00000293 | WB_rd=x0 WB_data=0x0000000000000008
WB_en=0
Cycle   8 | PC= 28 | IF_Instr=0x00000033 | IF_ID_Instr=0x0100b203 | WB_rd=x3 WB_data=0x0000000000000063
WB_en=1
Cycle   9 | PC= 32 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x16 WB_data=0x0000000000000018
WB_en=0
Cycle  10 | PC= 36 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x5 WB_data=0x0000000000000000
WB_en=1
Cycle  11 | PC= 40 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x4 WB_data=0x0000000000000063
WB_en=1
========================================
Pipeline execution complete at cycle 12
Total cycles: 12
pipe_bonus_tb.v:20: $finish called at 145000 (1ps)
```

Figure 20: Output

The store instruction requires the value loaded into `x3`. The value from the load instruction becomes available in the MEM/WB pipeline register. Therefore, the loaded data is forwarded directly from MEM/WB to the memory stage of the store instruction.

Forwarding condition:

if $(MEM/WB.RegWrite = 1)$ $\&(MEM/WB.RegisterRd = ID/EX.RegisterRs2)$ then $ForwardStore = 01$

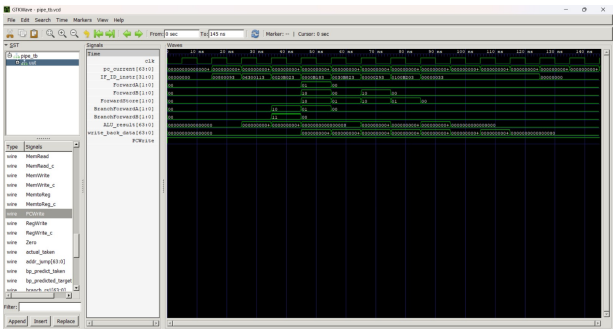

Figure 21: GTK Wave

5.6.7 TestCase 7:

```
addi x1, x0, 0
addi x0, x0, 0
beq  x1, x0, 8
addi x5, x0, 77
addi x6, x0, 88
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation
Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00000093 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x00000013 | IF_ID_Instr=0x00000093 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x00008463 | IF_ID_Instr=0x00000013 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x04d00293 | IF_ID_Instr=0x00008463 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x05800313 | IF_ID_Instr=0x00000033 | WB_rd=x1 WB_data=0x0000000000000000
WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00000033 | IF_ID_Instr=0x05800313 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=1
Cycle   7 | PC= 24 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   8 | PC= 28 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=1
Cycle   9 | PC= 32 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x6 WB_data=0x0000000000000058
WB_en=1
========================================
Pipeline execution complete at cycle 10
Total cycles: 10
pipe_bonus_tb.v:20: $finish called at 125000 (1ps)
```

Figure 22: Output

The branch instruction requires the value of `x1` as the first source register (`rs1`). At this time the result of the previous instruction is available in the EX/MEM pipeline register. Therefore, the value is forwarded from EX/MEM to the branch comparator in the ID stage.

Forwarding condition:

if $(EX/MEM.RegWrite = 1)$ $\&(EX/MEM.RegisterRd = IF/ID.RegisterRs1)$ then $BranchForwardA = 10$

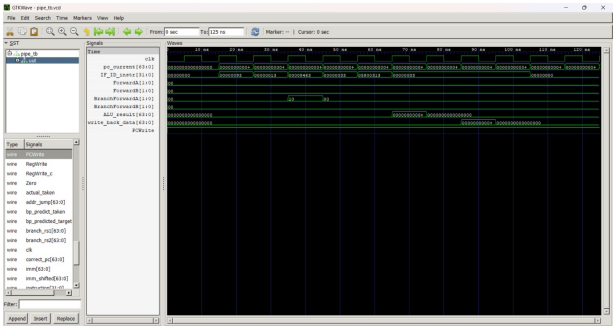

Figure 23: GTK Wave

5.6.8 TestCase 8:

```
addi x1, x0, 0
addi x0, x0, 0
beq  x0, x1, 8
addi x5, x0, 77
addi x6, x0, 88
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation
Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00000093 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x00000013 | IF_ID_Instr=0x00000093 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x00100463 | IF_ID_Instr=0x00000013 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x04d00293 | IF_ID_Instr=0x00100463 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x05800313 | IF_ID_Instr=0x00000033 | WB_rd=x1 WB_data=0x0000000000000000
WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00000033 | IF_ID_Instr=0x05800313 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=1
Cycle   7 | PC= 24 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   8 | PC= 28 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=1
Cycle   9 | PC= 32 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x6 WB_data=0x0000000000000058
WB_en=1
========================================
Pipeline execution complete at cycle 10
Total cycles: 10
```

Figure 24: Output

The branch instruction requires the value of `x1` as the second source register (`rs2`). The value of `x1` produced by the previous instruction is available in the EX/MEM pipeline register. Therefore, the value is forwarded from EX/MEM to the branch comparator in the ID stage.

Forwarding condition:

if $(EX/MEM.RegWrite = 1)$ $\&(EX/MEM.RegisterRd = IF/ID.RegisterRs2)$ then $BranchForwardB = 10$

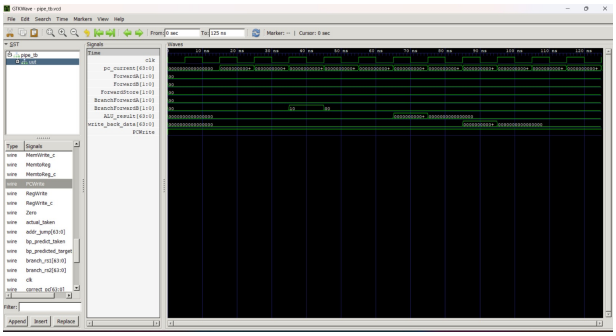

Figure 25: GTK Wave

5.6.9 TestCase 9:

```
addi x1, x0, 0
```

```
addi x0, x0, 0
addi x0, x0, 0
beq  x1, x0, 8
addi x5, x0, 77
addi x6, x0, 88
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

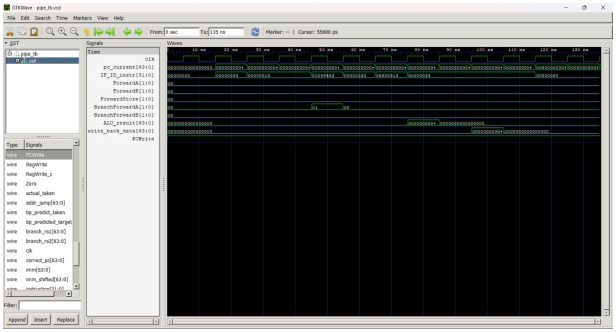

Figure 27: GTK Wave

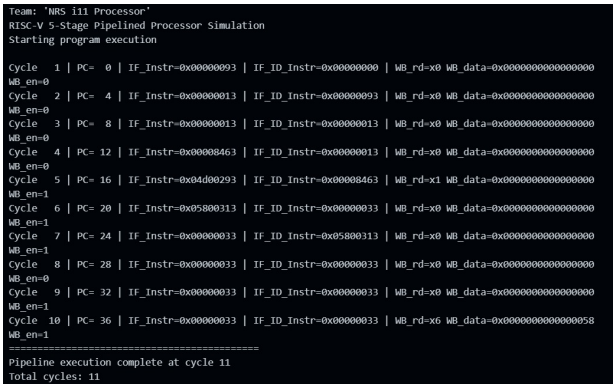

Figure 26: Output

The branch instruction requires the value of `x1` as the first source register (`rs1`). The value of `x1` is produced by the first instruction and is available in the MEM/WB pipeline register when the branch instruction reaches the ID stage. Therefore, the value is forwarded from MEM/WB to the branch comparator in the ID stage.

Forwarding condition:

if $(MEM/WB.RegWrite = 1)$ $\&(MEM/WB.RegisterRd = IF/ID.RegisterRs1)$ then $BranchForwardA = 01$

5.6.10 TestCase 10:

```
addi x1, x0, 0
addi x0, x0, 0
addi x0, x0, 0
beq  x0, x1, 8
addi x5, x0, 77
addi x6, x0, 88
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

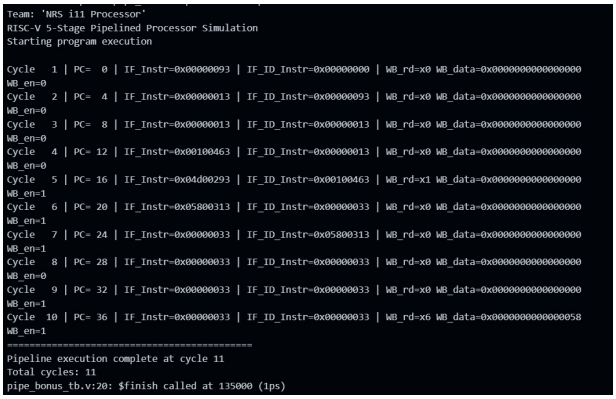

Figure 28: Output

The branch instruction requires the value of `x1` as the second source register (`rs2`). The value of `x1` produced by the first instruction is available in the

MEM/WB pipeline register when the branch instruction reaches the ID stage. Therefore, the value is forwarded from MEM/WB to the branch comparator in the ID stage.

Forwarding condition:

if $(MEM/WB.RegWrite = 1)$ &$(MEM/WB.RegisterRd = IF/ID.RegisterRs2)$ then $BranchForwardB = 01$

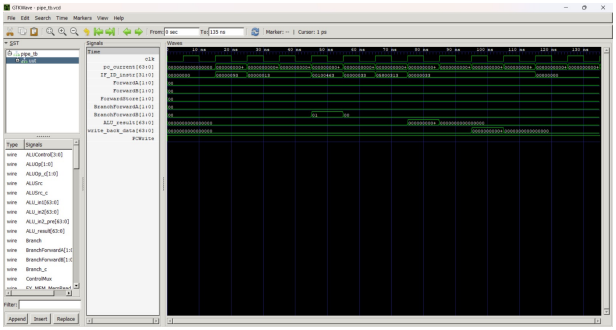

Figure 29: GTK Wave

5.6.11 TestCase 11:

```
addi x1, x0, 0
beq  x1, x0, 8
addi x5, x0, 77
addi x6, x0, 88
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation
Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00000093 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x00008463 | IF_ID_Instr=0x00000093 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x04d00293 | IF_ID_Instr=0x00008463 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x05800313 | IF_ID_Instr=0x00000033 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x00000033 | IF_ID_Instr=0x05800313 | WB_rd=x1 WB_data=0x0000000000000000
WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x1 WB_data=0x0000000000000000
WB_en=0
Cycle   7 | PC= 24 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=1
Cycle   8 | PC= 28 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x6 WB_data=0x0000000000000058
WB_en=1
========================================
Pipeline execution complete at cycle 9
Total cycles: 9
pipe_bonus_tb.v:20: $finish called at 115000 (1ps)
```

Figure 30: Output

The branch instruction requires the value of `x1` as the first source register (`rs1`). The value of `x1` is produced by the previous ALU instruction and is available in the EX stage when the branch instruction reaches the ID stage. Therefore, the ALU result is forwarded directly from the EX stage to the branch comparator in the ID stage.

Forwarding condition:

if $(EX.RegWrite = 1)$ &$(EX.RegisterRd = IF/ID.RegisterRs1)$ then $BranchForwardA = 11$

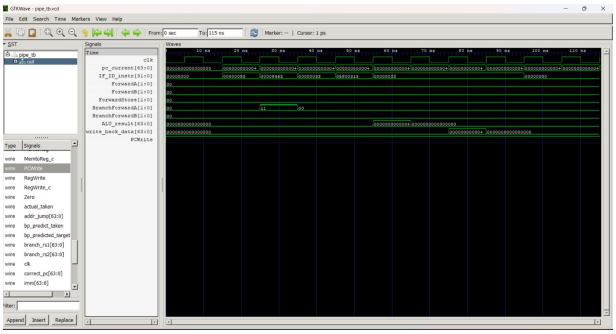

Figure 31: GTK Wave

5.6.12 TestCase 12:

```
addi x1, x0, 0
beq  x0, x1, 8
addi x5, x0, 77
addi x6, x0, 88
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

Figure 32: Output

The branch instruction requires the value of `x1` as the second source register (`rs2`). The value of `x1` is produced by the previous ALU instruction and is available in the EX stage when the branch instruction reaches the ID stage. Therefore, the ALU result is forwarded directly from the EX stage to the branch comparator in the ID stage.

Forwarding condition:
if $(EX.RegWrite = 1)$
$\&(EX.RegisterRd = IF/ID.RegisterRs2)$
then $BranchForwardB = 11$

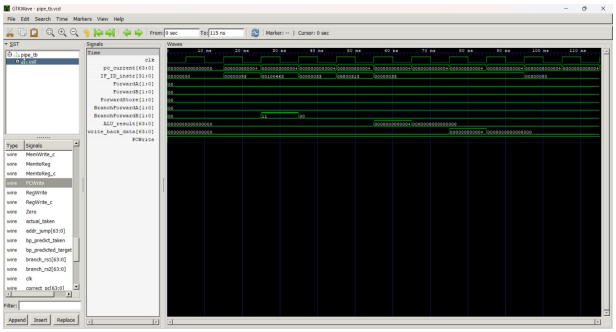

Figure 33: GTK Wave

5.6.13 TestCase 13:

```
addi x1, x0, 8
addi x2, x0, 42
sd   x2, 0(x1)
ld   x3, 0(x1)
add  x4, x3, x0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

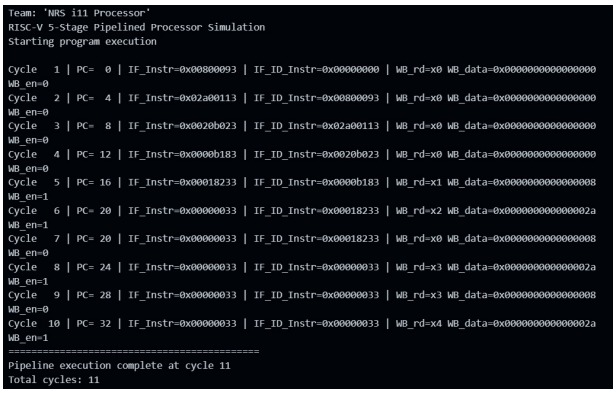

Figure 34: Output

Scenario: The instruction `ld x3, 0(x1)` loads a value into register `x3`. The next instruction `add x4, x3, x0` immediately uses this value as a source operand. Since the loaded value becomes available only after the MEM stage, the Hazard Detection Unit inserts a one-cycle stall. After the stall, the value is forwarded from the MEM/WB pipeline register to the EX stage. **Test Case: Load-Use Data Hazard (1-cycle stall)**

Scenario: The instruction `ld x3, 0(x1)` loads a value into register `x3`. The next instruction `add x4, x3, x0` immediately uses this value as a source operand. Since the loaded value becomes available only after the MEM stage, the Hazard Detection Unit inserts a one-cycle stall. After the stall, the value is forwarded from the MEM/WB pipeline register to the EX stage.

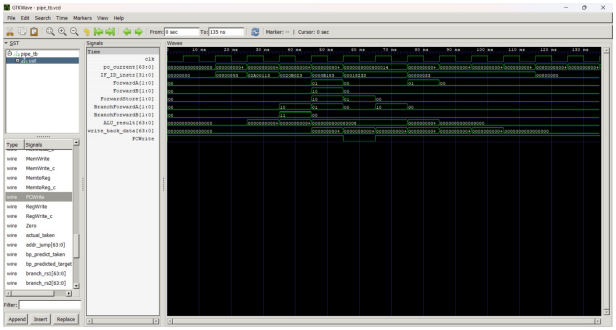

Figure 35: GTK Wave

5.6.14 TestCase-14:

```
    addi x1, x0, 5
addi x1, x0, 10
add  x2, x1, x0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
addi x0, x0, 0
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation
Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00500093 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x00a00093 | IF_ID_Instr=0x00500093 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x00008133 | IF_ID_Instr=0x00a00093 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x00000033 | IF_ID_Instr=0x00008133 | WB_rd=x0 WB_data=0x0000000000000000
WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x1 WB_data=0x0000000000000005
WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x1 WB_data=0x000000000000000a
WB_en=1
Cycle   7 | PC= 24 | IF_Instr=0x00000033 | IF_ID_Instr=0x00000033 | WB_rd=x2 WB_data=0x000000000000000a
WB_en=1
========================================
Pipeline execution complete at cycle 8
Total cycles: 8
pipe_bonus_tb.v:20: $finish called at 105000 (1ps)
```

Figure 36: Output

Double Data Hazard

Two consecutive instructions write to the same register x1. When the third instruction executes, both EX/MEM and MEM/WB pipeline registers match the source register. The forwarding unit selects the most recent value from the EX/MEM stage.

Forwarding condition:

if (EX/MEM.RegWrite = 1) & (EX/MEM.RegisterRd $\neq$ 0)&($EX/MEM.RegisterRd = ID/EX.RegisterRs$)

then ForwardA = 10

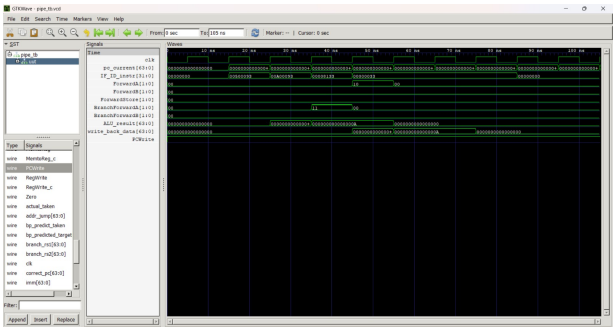

Figure 37: GTK Wave

6 Hazard Detection Unit

The Hazard Detection Unit is responsible for detecting data hazards that cannot be resolved using forwarding. In particular, it detects the **load-use hazard**, which occurs when an instruction immediately following a load instruction requires the loaded data.

Consider the following instruction sequence:

```
lw  x2, 0(x1)
add x4, x2, x5
```

In this case, the load instruction retrieves data from memory and stores it in register `x2`. However, the loaded data becomes available only after the Memory stage. The next instruction requires this value during its Execute stage. Since the data is not yet available, forwarding alone cannot resolve the dependency.

To handle this situation, the Hazard Detection Unit stalls the pipeline for one clock cycle by inserting a bubble. During this stall cycle:

- The Program Counter (PC) is not updated.
- The IF/ID pipeline register is not updated.
- A bubble instruction is inserted into the pipeline.

This delay allows the load instruction to complete the memory access so that the correct data becomes available for the dependent instruction.

The hazard detection condition is given by:

if $(ID/EX.MemRead = 1)$
&$((ID/EX.RegisterRd = IF/ID.RegisterRs1)$
or $(ID/EX.RegisterRd = IF/ID.RegisterRs2))$

When this condition is satisfied, the pipeline control signals are modified to stall the pipeline and prevent incorrect execution.

7 Control Hazard using 2-bit Branch Predictor

Control hazards occur when the processor encounters a branch instruction and the next instruction to be executed depends on the branch outcome. If the processor waits until the branch is resolved, the pipeline would stall and reduce performance. To avoid this, a **2-bit branch predictor** is used.

The predictor uses a **2-bit saturating counter** stored in a Branch History Table (BHT). Each branch instruction is associated with a counter that records the recent behavior of that branch. Based on the counter value, the processor predicts whether the branch will be taken or not taken.

The four states of the 2-bit predictor are:

State	Meaning
00	Strongly Not Taken
01	Weakly Not Taken
10	Weakly Taken
11	Strongly Taken

If the counter value is `00` or `01`, the predictor predicts that the branch will **not be taken**. If the value is `10` or `11`, the predictor predicts that the branch will **be taken**.

When the branch instruction reaches the Execute stage and the actual branch outcome is known, the counter is updated as follows:

- If the branch is taken, the counter increments toward `11`.
- If the branch is not taken, the counter decrements toward `00`.
- The counter saturates at `00` and `11`, meaning it cannot move beyond these states.

If the prediction matches the actual branch result, the pipeline continues execution without any penalty. However, if the prediction is incorrect, the pipeline must flush the incorrectly fetched instruction and update the program counter with the correct target address.

Using a 2-bit predictor improves performance because it avoids changing the prediction due to a single unusual branch outcome and instead adapts based on the recent history of the branch.

7.1 Early Branch Resolution and Branch Forwarding

In a conventional pipelined processor, the branch decision is resolved in the **Execute (EX)** stage. When a branch instruction is fetched, the processor predicts the branch direction and continues fetching the next sequential instruction. If the prediction is incorrect, the incorrectly fetched instruction must be flushed from the pipeline. Since the branch outcome is known only in the EX stage, the misprediction penalty is typically one cycle.

In this design, an additional **branch comparator is introduced in the Instruction Decode (ID) stage**. By performing the comparison earlier in the pipeline, the processor can determine the branch outcome sooner. As a result, the misprediction can be detected earlier and the number of wasted cycles in the pipeline is reduced.

However, moving the branch comparison to the ID stage introduces a new challenge. The branch instruction may depend on the result of a previous instruction whose value has not yet been written back to the register file. To handle this situation, **branch forwarding logic** is implemented.

Two forwarding multiplexers, **ForwardA** and **ForwardB**, are added at the inputs of the branch comparator in the ID stage. These multiplexers allow the most recent values of the source registers

to be forwarded from later pipeline stages, such as the **EX/MEM** or **MEM/WB** pipeline registers, directly to the comparator.

By introducing an early branch comparator in the ID stage and adding forwarding multiplexers for the branch operands, the processor can resolve branch conditions earlier while still avoiding data hazards. This design improvement reduces the effective branch penalty and improves overall pipeline efficiency.

7.2 TestCases

7.2.1 Testcase 1:

```
addi x1, x0, 5
addi x2, x0, 10
beq  x1, x2, 8
addi x3, x0, 99
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation

Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00500093 | IF_ID_Instr=0x00000000 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x00a00113 | IF_ID_Instr=0x00500093 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x00208463 | IF_ID_Instr=0x00a00113 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x06300193 | IF_ID_Instr=0x00208463 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x00000000 | IF_ID_Instr=0x06300193 | WB_rd=x1
WB_data=0x0000000000000005 WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x2
WB_data=0x000000000000000a WB_en=1
Cycle   7 | PC= 24 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x8
WB_data=0xfffffffffffffffb WB_en=0
Cycle   8 | PC= 28 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x3
WB_data=0x0000000000000063 WB_en=1
Cycle   9 | PC= 32 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0

========================================
Pipeline execution complete at cycle 9
```

Figure 38: output

The branch instruction compares the values in `x1` and `x2`. Since `x1 = 5` and `x2 = 10`, the condition `x1 = x2` is false and the branch is not taken. The processor uses a static branch prediction strategy that assumes branches are not taken. In this case the prediction is correct, so the pipeline continues fetching sequential instructions. Therefore, the instruction `addi x3, x0, 99` is executed normally and no pipeline flush or stall occurs. As a result, there is no penalty cycle in the pipeline.

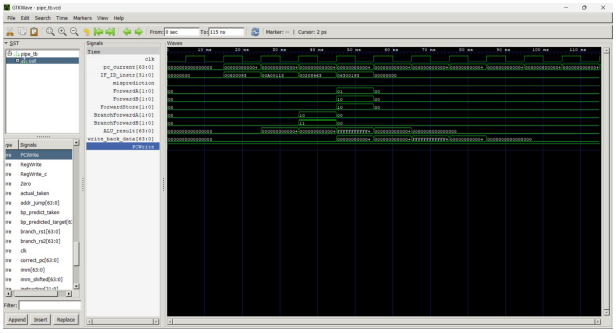

Figure 39: GTK Wave

7.2.2 Testcase 2:

```
addi x1, x0, 5
addi x2, x0, 5
beq  x1, x2, 8
addi x3, x0, 77
addi x4, x0, 88
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation

Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00500093 | IF_ID_Instr=0x00000000 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x00500113 | IF_ID_Instr=0x00500093 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x00208463 | IF_ID_Instr=0x00500113 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x04d00193 | IF_ID_Instr=0x00208463 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x05800213 | IF_ID_Instr=0x00000000 | WB_rd=x1
WB_data=0x0000000000000005 WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00000000 | IF_ID_Instr=0x05800213 | WB_rd=x2
WB_data=0x0000000000000005 WB_en=1
Cycle   7 | PC= 24 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x2
WB_data=0x0000000000000000 WB_en=0
Cycle   8 | PC= 28 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0
Cycle   9 | PC= 32 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x4
WB_data=0x0000000000000058 WB_en=1
Cycle  10 | PC= 36 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x0
WB_data=0x0000000000000000 WB_en=0

========================================
Pipeline execution complete at cycle 10
```

Figure 40: Output

The branch instruction compares the values in `x1` and `x2`. Since both registers contain the value 5, the condition `x1 = x2` is true and the branch is taken. However, the processor uses a static branch prediction strategy that assumes branches are not taken. As a result, the next sequential instruction `addi x3,`

`x0, 77` is fetched incorrectly. When the branch decision is resolved in the EX stage, this incorrect instruction is flushed from the pipeline. The processor then begins fetching instructions from the correct branch target address. This causes a one-cycle penalty due to the pipeline flush.

Control condition:

if $(Branch = 1)$
& $(Zero = 1)$
then the pipeline is flushed and the PC is updated to the branch target.

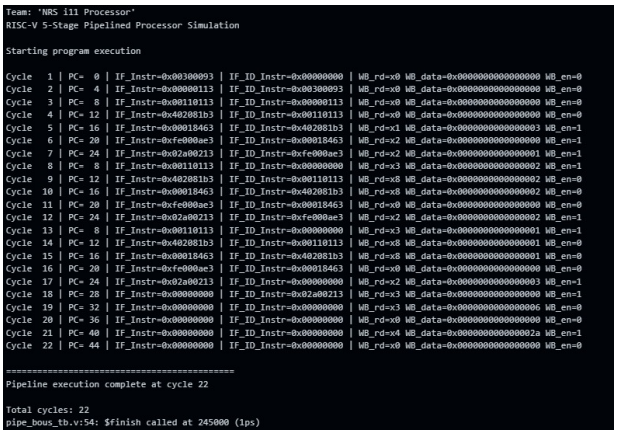

Figure 42: Output

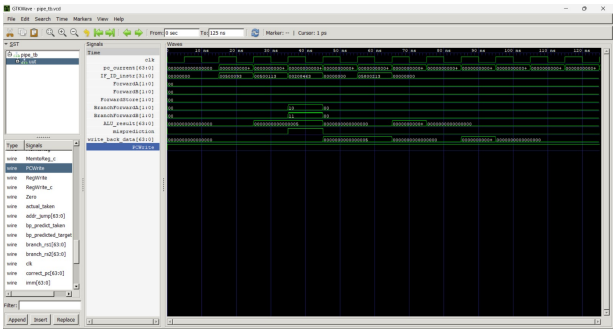

Figure 41: GTK Wave

This test case demonstrates how the branch predictor learns from repeated executions of a loop. The register `x1` stores the loop count and `x2` acts as the loop counter. During each iteration, `x2` is incremented and compared with `x1`. If the values are not equal, execution jumps back to the loop body. When the values become equal, the branch condition is satisfied and the loop exits.

7.2.3 Testcase 3:

```
addi x1, x0, 3
addi x2, x0, 0
addi x2, x2, 1
sub  x3, x1, x2
beq  x3, x0, 8
beq  x0, x0, -12
addi x4, x0, 42
```

Initially the branch predictor starts in the cold state and predicts the branch as not taken. During the first two iterations the branch is actually taken, resulting in mispredictions and a one-cycle penalty. After observing these outcomes, the predictor updates its internal counter and begins predicting the branch as taken. On the third iteration the prediction is correct and no penalty occurs. When the loop exits, the predictor again mispredicts because it expects the branch to be taken.

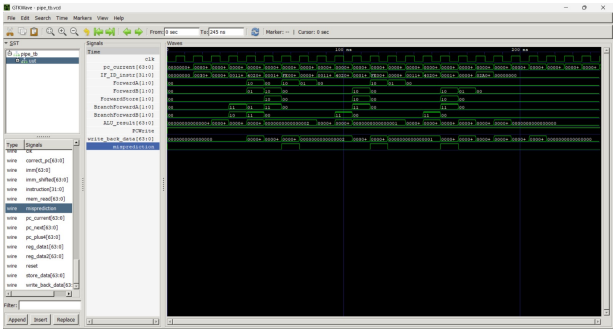

Figure 43: GTK Wave

7.2.4 Testcase 4:

```
addi x1, x0, 0
beq  x1, x0, 8
addi x5, x0, 77
addi x1, x0, 0
beq  x1, x0, 8
addi x5, x0, 77
addi x1, x0, 0
beq  x1, x0, 8
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation

Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00000093 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x00008463 | IF_ID_Instr=0x00000093 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x04d00293 | IF_ID_Instr=0x00008463 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x00000693 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x00008863 | IF_ID_Instr=0x00000693 | WB_rd=x1 WB_data=0x0000000000000000 WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x04d00293 | IF_ID_Instr=0x00008863 | WB_rd=x1 WB_data=0x0000000000000000 WB_en=0
Cycle   7 | PC= 32 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle   8 | PC= 36 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x13 WB_data=0x0000000000000000 WB_en=1
Cycle   9 | PC= 40 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x13 WB_data=0x0000000000000000 WB_en=0

========================================
Pipeline execution complete at cycle 9

Total cycles: 9
pipe_bous_tb.v:54: $finish called at 115000 (1ps)
```

Figure 44: Output

This test case demonstrates a correctly predicted taken branch after the branch predictor has been trained. The register `x1` is initialized to zero and compared with `x0`. Since both values are equal, the branch condition is true and the branch is taken.

Initially, the branch predictor may mispredict because it starts in the cold state. However, after executing the same branch multiple times, the predictor updates its internal counter and begins predicting the branch as taken. When the prediction matches the actual outcome, the pipeline continues execution without flushing any instructions.

Therefore, the branch is executed with zero penalty cycles, representing the best-case scenario for branch prediction in the pipeline.

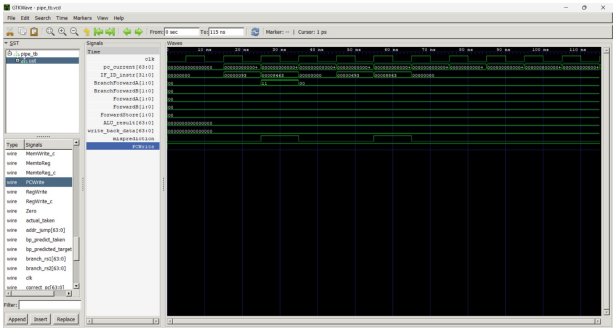

Figure 45: GTK Wave

7.2.5 Testcase 5:

```
addi x1, x0, 3
addi x2, x0, 1
addi x3, x0, 0
add  x3, x3, x2
sub  x4, x1, x3
beq  x4, x0, 8
beq  x0, x0, -12
addi x5, x0, 42
```

```
Team: 'NRS i11 Processor'
RISC-V 5-Stage Pipelined Processor Simulation

Starting program execution

Cycle   1 | PC=  0 | IF_Instr=0x00300093 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle   2 | PC=  4 | IF_Instr=0x00100113 | IF_ID_Instr=0x00300093 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle   3 | PC=  8 | IF_Instr=0x00000193 | IF_ID_Instr=0x00100113 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle   4 | PC= 12 | IF_Instr=0x002181b3 | IF_ID_Instr=0x00000193 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle   5 | PC= 16 | IF_Instr=0x40308233 | IF_ID_Instr=0x002181b3 | WB_rd=x1 WB_data=0x0000000000000003 WB_en=1
Cycle   6 | PC= 20 | IF_Instr=0x00020463 | IF_ID_Instr=0x40308233 | WB_rd=x2 WB_data=0x0000000000000001 WB_en=1
Cycle   7 | PC= 24 | IF_Instr=0xfe000ae3 | IF_ID_Instr=0x00020463 | WB_rd=x3 WB_data=0x0000000000000000 WB_en=1
Cycle   8 | PC= 28 | IF_Instr=0x02a00293 | IF_ID_Instr=0xfe000ae3 | WB_rd=x3 WB_data=0x0000000000000001 WB_en=1
Cycle   9 | PC= 12 | IF_Instr=0x002181b3 | IF_ID_Instr=0x00000000 | WB_rd=x4 WB_data=0x0000000000000002 WB_en=1
Cycle  10 | PC= 16 | IF_Instr=0x40308233 | IF_ID_Instr=0x002181b3 | WB_rd=x8 WB_data=0x0000000000000002 WB_en=0
Cycle  11 | PC= 20 | IF_Instr=0x00020463 | IF_ID_Instr=0x40308233 | WB_rd=x8 WB_data=0x0000000000000002 WB_en=0
Cycle  12 | PC= 24 | IF_Instr=0xfe000ae3 | IF_ID_Instr=0x00020463 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle  13 | PC= 28 | IF_Instr=0x02a00293 | IF_ID_Instr=0xfe000ae3 | WB_rd=x3 WB_data=0x0000000000000002 WB_en=1
Cycle  14 | PC= 12 | IF_Instr=0x002181b3 | IF_ID_Instr=0x00000000 | WB_rd=x4 WB_data=0x0000000000000001 WB_en=1
Cycle  15 | PC= 16 | IF_Instr=0x40308233 | IF_ID_Instr=0x002181b3 | WB_rd=x8 WB_data=0x0000000000000001 WB_en=0
Cycle  16 | PC= 20 | IF_Instr=0x00020463 | IF_ID_Instr=0x40308233 | WB_rd=x8 WB_data=0x0000000000000001 WB_en=0
Cycle  17 | PC= 24 | IF_Instr=0xfe000ae3 | IF_ID_Instr=0x00020463 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle  18 | PC= 28 | IF_Instr=0x02a00293 | IF_ID_Instr=0x00000000 | WB_rd=x3 WB_data=0x0000000000000003 WB_en=1
Cycle  19 | PC= 32 | IF_Instr=0xfe000000 | IF_ID_Instr=0x02a00293 | WB_rd=x4 WB_data=0x0000000000000000 WB_en=1
Cycle  20 | PC= 36 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x4 WB_data=0x0000000000000006 WB_en=0
Cycle  21 | PC= 40 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0
Cycle  22 | PC= 44 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x5 WB_data=0x000000000000002a WB_en=1
Cycle  23 | PC= 48 | IF_Instr=0x00000000 | IF_ID_Instr=0x00000000 | WB_rd=x0 WB_data=0x0000000000000000 WB_en=0

========================================
Pipeline execution complete at cycle 23
```

Figure 46: Output

This test case demonstrates a branch misprediction after the branch predictor has been trained to predict the branch as *taken*. The loop executes multiple iterations, causing the predictor counter to move toward the strongly taken state.

During the final iteration, the branch condition becomes false and the actual outcome is *not taken*. However, since the predictor has been trained to predict taken, it incorrectly predicts the branch as taken. As a result, the pipeline must flush the incorrectly fetched instruction and redirect the program counter to the correct address.

Therefore, this scenario produces a branch misprediction with a pipeline flush penalty.

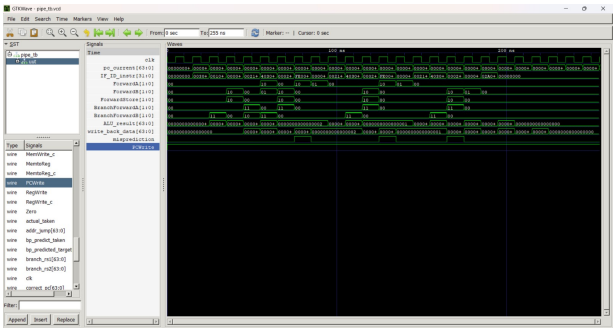

Figure 47: GTK Wave

8 Fibonacci Series

```
addi x1, x0, 1        # x1 = 1 (fib_prev = fib[1])
addi x2, x0, 1        # x2 = 1 (fib_curr = fib[2])
addi x4, x0, 0        # x4 = 0 (base memory address)
addi x5, x0, 8        # x5 = 8 (loop counter: 8 more terms after first 2)
addi x6, x0, 1        # x6 = 1 (constant 1 for counter)
addi x8, x0, 0        # x8 = 0 (current memory offset)

sd   x1, 0(x4)        # mem[0] = 1 (fib[1])
sd   x2, 8(x4)        # mem[8] = 1 (fib[2])

addi x8, x0, 16       # x8 = 16 (next store offset = mem[16])

add  x3, x1, x2       # x3 = fib_prev + fib_curr = next fib
add  x9, x4, x8       # x9 = base + offset (store address)
sd   x3, 0(x9)        # mem[x9] = x3 (store fib[n])

or   x1, x2, x0       # x1 = x2 | 0 = x2 (fib_prev = fib_curr)
or   x2, x3, x0       # x2 = x3 | 0 = x3 (fib_curr = fib_next)

addi x8, x8, 8        # x8 = x8 + 8 (advance offset)
sub  x5, x5, x6       # x5 = x5 - 1
beq  x5, x0, 8        # if counter==0: exit loop (skip back-branch)
beq  x0, x0, -32      # unconditional: jump back to loop start

ld   x10, 0(x4)       # x10 = mem[0]  = fib[1]  = 1
ld   x11, 8(x4)       # x11 = mem[8]  = fib[2]  = 1
ld   x12, 16(x4)      # x12 = mem[16] = fib[3]  = 2
ld   x13, 24(x4)      # x13 = mem[24] = fib[4]  = 3
ld   x14, 32(x4)      # x14 = mem[32] = fib[5]  = 5
ld   x15, 40(x4)      # x15 = mem[40] = fib[6]  = 8
ld   x16, 48(x4)      # x16 = mem[48] = fib[7]  = 13
ld   x17, 56(x4)      # x17 = mem[56] = fib[8]  = 21
ld   x18, 64(x4)      # x18 = mem[64] = fib[9]  = 34
ld   x19, 72(x4)      # x19 = mem[72] = fib[10] = 55

add  x20, x18, x19    # x20 = 34 + 55 = 89 = fib[11]

beq  x10, x11, 8      # taken: fib[1]==fib[2]==1, skip next
addi x21, x0, 999     # SKIPPED (x21 stays 0)

and  x22, x10, x11    # x22 = 1 & 1 = 1 (both fib[1]=fib[2]=1)
sub  x23, x19, x18    # x23 = 55 - 34 = 21 = fib[8] (Fib property check)
add x0, x0, x0          # dummy instructions to clear pipeline
add x0, x0, x0
add x0, x0, x0
add x0, x0, x0
```

Figure 48: Instruction Set

register_file.txt

```
1  0000000000000000
2  0000000000000022
3  0000000000000037
4  0000000000000037
5  0000000000000000
6  0000000000000000
7  0000000000000001
8  0000000000000000
9  0000000000000050
10 0000000000000048
11 0000000000000001
12 0000000000000001
13 0000000000000002
14 0000000000000003
15 0000000000000005
16 0000000000000008
17 000000000000000d
18 0000000000000015
19 0000000000000022
20 0000000000000037
21 0000000000000059
22 0000000000000000
23 0000000000000001
24 0000000000000015
25 0000000000000000
26 0000000000000000
27 0000000000000000
28 0000000000000000
29 0000000000000000
30 0000000000000000
31 0000000000000000
32 0000000000000000
33 105
34
```

Figure 49: Register File

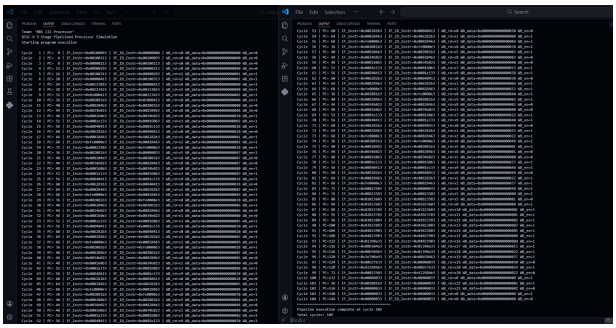

Figure 50: Output

9 Bonus

As an additional enhancement, extra forwarding units and multiplexers were introduced at the inputs of the branch comparator in the **ID stage** to support branch operand forwarding.

The control hazard handling mechanism was also improved by replacing the static **not-taken** prediction with a **2-bit branch predictor**. Furthermore, by placing the branch comparator in the **ID stage**, the misprediction penalty was reduced from two stall cycles to **one stall cycle**, improving overall pipeline efficiency.

10 Contribution

Ganipisetty Nischal:

- Implemented the Instruction Decode (ID) stage.
- Designed the register memory for storing and retrieving operand values.
- Developed the Execute (EX) stage for performing arithmetic and logic operations.
- Implemented the branch hazard unit to handle control hazards in the pipeline.
- Generated test cases and performed debugging of the processor.

Batta Venkata Shashank:

- Implemented the Data Memory (MEM) stage for load (`ld`) and store (`sd`) operations for both sequential and pipelined processors.
- Designed the Immediate Generator for handling immediate values in instructions for both sequential and pipelined processors.
- Developed the Write-Back (WB) stage for updating register values for both sequential and pipelined processors.
- Developed the Forwarding Unit to resolve data hazards efficiently.
- Prepared the final report, documenting the design, implementation, and results.

Regalla Kowshikh Reddy:

- Designed the Control Unit for both sequential and pipelined execution and hazard handling.
- Implemented the Program Counter (PC) for instruction sequencing.
- Developed the Instruction Fetch (IF) stage for fetching instructions from memory.
- Developed the Hazard Detection Unit to manage pipeline stalls.
- Managed integration and modularity in both sequential and pipelined implementations.
- Generated test cases and instruction sequences to verify correct execution.
- Developed testbenches for all modules.